

EvolveSWDB weekly update

From a software database to evaluating BFS rewrites

Yan-Ru Jhou 2026-09-30

Where we are

Last week

Built the Software Database

- Real code + memory access patterns
- Inputs, machine, measured profiles
- Query and export for the agents

This week

Use it to evaluate code rewrites

- Agent proposes a change to BFS
- We build, check, and measure it
- Native CPU and DX100 simulation

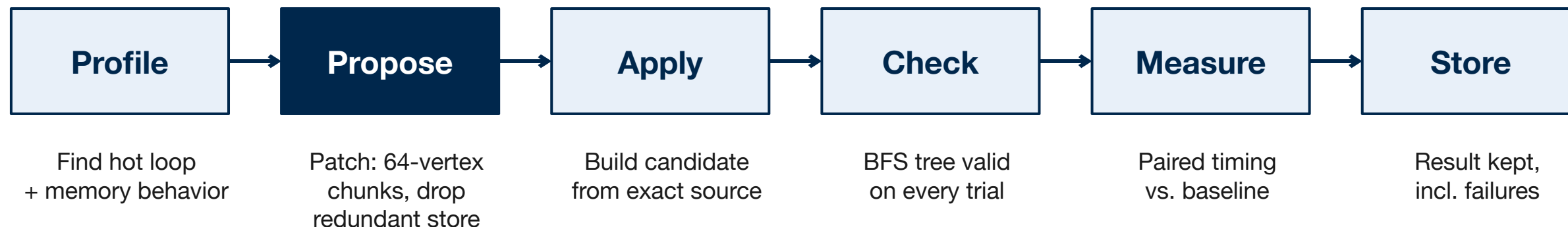
Question: can a rewrite proposal go from bottleneck to a trustworthy result?

Why an evaluation workflow?

	What it means for us
Motivation	Agents will propose rewrites; each needs a trustworthy verdict
Problem	Guesses and aggregate counters do not locate a workload's hot code
Problem	Two BFS origins: speedup is ambiguous without an explicit baseline
Solution	One workflow: profile → propose → apply → check → measure → store
Stand-in	Representative proposals until the live agents are ready

Failures and regressions are kept too, not only wins.

The workflow on one example



Two proposal forms

Instructions: natural language or structured steps

Code: a patch or annotated source

The system never picks its own optimization; it evaluates the one proposed.

What we have done this week

Area	Status
Workflow interfaces (profile, apply, check, measure, store)	Done
DX100 build, correctness, profiling support	Done
Native runs: DX100 patch route, upstream instruction route	Done
Remaining routes and DX100 simulation runs	In progress
Final coverage report and handoff	In progress

2 implementations × 2 proposal forms × 2 graph families = 8 cases

Acceptance: 11 of 20 criteria met. Test suite: 2,672 passed, 0 failed.

First native results: no measurable change

mbit10 Xeon Gold 6326, 1 thread, 10 paired repetitions, BFS region only

Proposal	Graph	Baseline (ms)	Candidate (ms)	Speedup	95% CI
DX100 BFS: patch	Uniform	46.03	45.96	1.002	[0.998, 1.011]
DX100 BFS: patch	Kronecker	23.26	23.23	1.000	[0.996, 1.020]
GAPBS BFS: instructions	Uniform	7.33	7.29	1.002	[0.999, 1.005]
GAPBS BFS: instructions	Kronecker	6.00	5.99	1.001	[0.999, 1.030]

Correct on all timed trials. Spread above the 10% limit, so the verdict is “inconclusive”.

Small edits, no gain. The workflow refuses to report noise as a speedup.

Ongoing work and what we learned

Challenge	What we did	Status
LLM proposals: timeouts, one bad patch	Added whole-file edit format	New candidate built next
DX100 simulations take hours	Parallel runs within one socket	Running
Shared host: 2 jobs max	Queue per socket lane	Ongoing
Harness overhead per trial	Cached record parsing: 8.7 s → 1.0 s	Fixed

Lesson: most of the cost is around the experiment, not in it.

Plan for this week

1

Finish DX100 reference and control simulations

Explicit baseline for accelerated results

2

Evaluate remaining routes, incl. DX100 candidates

All 8 cases covered

3

Coverage report + handoff to SW/HW ensembles

Agents use the workflow directly

Next question: do larger rewrites give measurable gains?

Backup: native measurement protocol

Setting	Value
Host	mbit10, Intel Xeon Gold 6326, one socket lane
Threads	1
Repetitions	10 paired (baseline / candidate alternating)
Claim rule	Speedup ≥ 1.05 and spread $\leq 10\%$
Correctness	BFS tree checked on every timed trial
Profiling	Region timers + Callgrind (no HW counters)